Principles of Cybersecurity

Toghrul Karimov
Max Planck Institute for Software Systems

Last updated in 03.2026

Contents

1 Preliminaries **1**
1.1 Basic concepts from information security . . . 1
1.2 The Dolev-Yao model . . . 2

2 Hash functions **2**
2.1 Definitions and properties . . . 2
2.2 Some applications of hash functions . . . 3
2.3 Homework . . . 3

3 Encryption **3**
3.1 Symmetric-key cryptography . . . 3
3.1.1 Block ciphers . . . 4
3.1.2 Stream ciphers . . . 4
3.2 How do we establish a shared key in the first place? . . . 5
3.3 Asymmetric-key cryptography . . . 5
3.4 Digital signatures . . . 5
3.5 Discussion and homework . . . 6
3.6 Quick one: message authentication codes (MAC) . . . 6

4 Access control in operating systems **7**

5 Basics of the internet **7**
5.1 IP addresses . . . 7
5.2 The network layer . . . 8
5.3 Client-server architecture . . . 8
5.4 Homework . . . 8

6 Linux (bash) programming **8**

1 Preliminaries

We write $x \,||\, y$ for the concatenation of two strings x and y.

1.1 Basic concepts from information security

The three core concepts are the following, called the "CIA Triad". *Confidentiality* means that information is only accessible only to individuals to whom it is meant to be accessible. *Integrity* means that information is accurate, i.e. has not been tampered with/altered. *Availability* means that the systems are available when they are needed. In addition, *authentication* means verifying the identity of the person performing an action, e.g. we can verify that the person sending the message is indeed the person with whom intend to be communicating. Observe that authentication implies integrity. *Non-repudiation* means that once an actor performs an action, he or she cannot deny having performed it.

1.2 The Dolev-Yao model

Throughout this course, we assume that the attacker is "as powerful as possible". In particular, the attacker owns the network that transmits the messages, can store a large amount of messages from the past, has the capacity to send any message to anyone, and knows all the implementation details of our protocols (e.g., exactly which encryption protocol we are using) except the information that is explicitly secret (i.e., the keys). We additionally assume that the attacker has access to a large (but not infinite) amount of computational power that is commensurate with how much computation can realistically be performed at the time of writing.

2 Hash functions

2.1 Definitions and properties

A *hash function* is a function f that takes a binary string of arbitrary length, and outputs a binary string of fixed length ℓ. For example, `SHA2` (SHA stands for "Secure Hashing Algorithm") always outputs a binary string of length 256. A desirable property of a hash function is that, for every bit length n, the set of all images of f on strings of length n should be uniformly distributed. That is, for every n and binary string y of length ℓ, the number of binary strings x of length n such that $f(x) = y$ should be close to $2^n/2^\ell$. (Recall that 2^n is the number of different strings of length n, and 2^ℓ is the number of possible outputs of f.) Outside of cryptography, hash functions are used in programming, most notably in *hash tables*.

An example of a hash function is $f(x) = x \bmod 2^\ell$, where we convert back and forth between numbers and their binary representations. The outputs of this f are, in fact, exactly uniformly distributed. Another example of a hash function is f that squares the input and returns the middle ℓ bits from the resulting binary representation.

A *collision* for f is two distinct inputs x_1, x_2 such that $f(x_1) = f(x_2)$. A *cryptographic hash function* is hash function with extra properties related to their safety. These are as follows.

1. Preimage resistance. The problem of finding, given a string y of length ℓ, a string x such that $f(x) = y$ is *computationally infeasible*.
2. Second preimage resistance. The problem of finding, given x_1, y such that $f(x_1) = y$, a string x_2 such that $f(x_2) = y$ is computationally infeasible.
3. It should be computationally infeasible to generate new collisions for f.

When the properties (1-3) are satisfied, then the function f should be essentially "random". In particular, making a small change to x should completely change $f(x)$.

For example, `SHA2` does satisfy the properties (1-3) above: in fact, not a single collision is publicly known for it. The earlier version `SHA1` (which produces 160-bit outputs) does not satisfy (3); in fact, it has been cracked in a stronger sense. A *chosen prefix attack* has been successfully implemented, meaning that it is computationally feasible, given x_1, x_2, find two strings z_1, z_2 such that $f(x_1 \,||\, z_1) = f(x_2 \,||\, z_2)$, where $x \,||\, z$ denotes the concatenation of x and z. Today, `SHA1` is not considered secure and not used any more (read `https://en.wikipedia.org/wiki/SHA-1#Attacks` and `https://security.googleblog.com/2017/02/announcing-first-sha1-collision.html`). The hash function $f(x) = x \bmod 2^\ell$ we gave earlier does not satisfy any of the properties (1-3) above, and should not be used in any cryptographic application.

A common way to build cryptographic hash functions is through the *Merkle-Damgård construction*. We illustrate it for `SHA2`. Let g be a *one-way* function (meaning that it is computationally infeasible to invert it) that takes two inputs a, b of bit lengths 256 and 512, respectively. Further let b_0 be a fixed string of length 256, called the *initialisation vector* of the hash function. To hash an input x, we first pad it (i.e., extend it in some pre-defined way) to a word x' whose length is a multiple of 512. Write $x' = a_0 \,||\, a_1 \,||\, ... \,||\, a_m$, where each a_i has length 512. To compute the hash, we perform

$$\begin{aligned} b_1 &= g(a_0, b_0) \\ b_2 &= g(a_1, b_1) \\ &\vdots \\ b_{m+1} &= g(a_m, b_m) \end{aligned}$$

and output $y = b_{m+1}$.

Successful attacks on hash functions typically exploit its (block) structure; this is an example of *cryptanalysis*. A *brute-force attack*, on the other hand, does not use any information about f: rather, it tries to find a collision by generating $(x, f(x))$ pairs until a collision is found. A brute-force attack on a hash function is also called a *birthday attack*. The name comes from the *birthday problem*, which is as follows. Suppose we choose n people randomly. (Think of a person as a binary string and their birthday as the corresponding hash.) What is the probability that two people share the same birthday? It turns out that for $n = 50$, this probability is already $\approx 97\%$, despite 50 being much smaller than the number 366 of all possible birthdays; see `https://en.wikipedia.org/wiki/Birthday_problem`. In the context of hash functions, by the same mathematical analysis, it is known that to find a collision for an ℓ-bit hash function, we need to generate $2^{\ell/2}$ pairs $(x, f(x))$ *on average*.

2.2 Some applications of hash functions

Hash functions have many applications in cryptography. We next discuss one of them: storing passwords. When you register an account on any website, nobody except you should now your passwords. Similarly, passwords should never ever be stored, due to the possibility of the database of passwords getting compromised. How can we then perform password-based authentication? The answer is: the authenticator stores not the password but its hash. When the user logs in, the hash of the entered string is computed and compared against the stored hash. Because cryptographic hash functions are preimage and collision-resistant, even if the database of stored password hashes gets stolen, the attacker will not be able to gain access to user accounts by breaking the hash function.

However, the protocol for storing passwords described above is still not secure. The reason is that the passwords used by real users are typically not random strings–rather, they often come from a relatively small list of "natural" strings. Hence an attacker crack at least some passwords by comparing the hashes in the password database against the hashes of all commonly used passwords (stored in an efficient, special data structure called *rainbow table*). Hence one should store *salted hashes* instead of just hashes. A salt is additional, *random* information that is added it to the password, which could, for example, be the date and time of password creation. That is, we store s and $f(p \,||\, s)$ on the database, where p is the password and s is the salt. This precludes the use of rainbow tables, even if the attacker knows all the salt values.

Another application of hash functions consists of *checksums*. When software is published, it usually comes with a way of checking the integrity of the files to be downloaded. See, for example, "File Checksums" in `https://www.virtualbox.org/wiki/Downloads`. When you download VirtualBox, it is your responsibility to compare the hash of the file you downloaded against the hash value provided by the publisher of the software: it is possible that an attacker has swapped (for example, somewhere in the network) the code you wanted to download with harmful code under the same file name.

2.3 Homework

Use `sha256sum` with your own examples. Look up `rockyou.txt`. Investigate the full capabilities of `hashcat`. Investigate what `openssl` is. Look up password hashing functions, e.g. `yescrypt` and `Argon2`.

3 Encryption

The goal of encryption is to send a message without revealing any of its contents to anyone other than the intended recipient. *Plaintext* refers to the message we want to send; we denote it by m. We use an *encryption key* (denoted k_e), to compute the *ciphertext* $c = f(m, k_e)$, where f is the *encryption algorithm*. Once the recipient receives the ciphertext, she *decrypts* it using the *decryption key* k_d: writing g for the *decryption algorithm*, we should have that $g(f(m, k_e), k_d) = m$ (as well as $f(g(m, k_d), k_e) = m$) for every possible string m and key pair k_e, k_d.

3.1 Symmetric-key cryptography

In this case, encryption and decryption are both performed using the same key $k = k_e = k_d$, which must be a secret between the sender and the intended recipient. There are two kinds of symmetric-key cryptographic algorithms: *block* and *stream* ciphers. In a block cipher, the plaintext is divided into

blocks, and each block is encrypted (and later decrypted) independently, all using the same secret key. On the other hand, in a stream cipher, we first generate a *keystream* s from the secret key that is as long as the plaintext, and then compute $m \oplus s$, i.e. the `XOR` of the plaintext and the keystream. We can always turn a block cipher into a stream cipher: the latter is a strictly large family of algorithms.

3.1.1 Block ciphers

The simplest class of symmetric-key encryption algorithms are the *substitution ciphers*. In this case, we replace each unit of plaintext (for example, every letter or every byte) with another one according to a prescribed rule. The secret key is then the (finite) list of replacement rules. *Caesar cipher* is an example of a substitution cipher. In a substitution cipher, both the encryption and decryption algorithms are the same. To see this, suppose we are working with strings of a, b, c, and encrypt plaintexts using the substitution $a \to b, b \to c, c \to a$. Then decryption with this key is the same as encryption with the key $a \to c, b \to a, c \to b$. Substitution ciphers can have a large number of possible keys (e.g., we can replace letters of the English alphabet according to $26! \approx 2^{88}$ different possible rules), but can be easily attacked using a *frequency analysis*. For example, we know that by far the most common letter in English is `e`. Therefore, if we have access to a large amount of ciphertext, even if we don't know the key k we can still determine the letter to which `e` is mapped by counting frequencies of letters in the ciphertexts. Once we identify the letter to which `e` is mapped, we can move on to another letter. See `https://en.wikipedia.org/wiki/Letter_frequency` for relative frequencies of each letter in English texts.

A more modern encryption algorithm is AES (Advanced Encryption Standard), standardised by NIST in 2001. It always divides the plaintext into blocks of 128 bits (padding the input to a multiple of 128 at the beginning if necessary), but can have 128, 196, or 256 bit keys. In this course, we work with AES256. Pure AES (referred to as AES in the *block mode*) operates on each 128-bit block b as follows, computing a 128-bit output. First, write $b = c_1 \,||\, \cdots \,||\, c_{16}$, where each c_i is a single byte. Then it arranges the 16 bytes into a 4×4 matrix. Thereafter, it performs 14 rounds of the following: perform a complicated but invertible operation on the matrix, derive a fresh round key k' from the secret key k, and take the `XOR` of the matrix with k'. No round key is re-used. Note that if we know k, then we know every round key k', and we can perform decryption because `XOR` with k' is an invertible operation: $b = (b \oplus k') \oplus k'$.

All pure block ciphers can be attacked using frequency analysis. Hence in practice, we always have some modification. AES itself has 4 other such modes: AES-GCM, AES-CBC, AES-CTR, AES-CCM.

3.1.2 Stream ciphers

We first give `AES-CTR` with 256 bit keys as example. We write $\mathrm{AES}(k, b)$ for the output of AES in the block mode (see the section above) on a 128-bit block b with key k. Again, we split the (padded) plaintext m into 128-bit blocks $b_1, \ldots, b_l$. Next, we generate a 96-bit random string r, called the *nonce* (number used only once). Thereafter, we generate the keystream

$$s = \mathrm{AES}(k, (r \,||\, 1)) \,||\, \cdots \,||\, \mathrm{AES}(k, (r \,||\, l))$$

which has exactly the same length m. Note that so far we have not used the plaintext at all: we have just encrypted the nonce together with the block numbers, called the *counters*. To produce the ciphertext, we then take the `XOR` of the keystream with the plaintext, i.e. $c = m \oplus s$. Importantly, we transmit the ciphertext c together with the nonce r: the latter is necessary to decrypt the message. (Compare the nonce to the salt used in hashing.) The receiver, upon receiving (c, r), first computes the keystream s from the shared key k and the nonce c, and finally computes

$$m = c \oplus s = (m \oplus s) \oplus s.$$

If the same nonce is re-used with the same key, then we are back to being vulnerable to frequency analysis. However, observe that the space of possible nonces is large with size 2^{96}.

Other modes of AES can provide *authentication* in addition to *confidentiality*. The AES-GCM mode, for example, transmits three pieces of data: the ciphertext, the nonce, and a 128-bit *tag*. The tag is computed from the ciphertext, the nonce, and the secret key using a hashing algorithm called GHASH. We can then verify the identity of the sender by comparing the tag against $\mathrm{GHASH}(c, r, k)$. This is called Authenticated Encryption with Associated Data (AEAD).

3.2 How do we establish a shared key in the first place?

Historically, shared secrets between two or more parties were established through separate safe channels: for example, the key was printed on a piece of paper, put into a sealed envelope, and then personally delivered to the other party. (See `https://en.wikipedia.org/wiki/Cryptanalysis_of_the_Enigma#Key_setting`.) However, using some interesting mathematics it is possible for two people to establish a secret key while communicating only over a public channel. We will discuss the *Diffie-Hellman key exchange protocol*, first published in 1976.

The protocol works as follows. Suppose Alice and Bob want to establish a secret k of length l bits. They first publicly agree on a prime number p of bit length l, and a base $1 < g < p$. Then Alice generates a secret number $0 \leq a < p$, computes $A := g^a \bmod p$, and transmits A to Bob. Similarly, Bob generates a secret number $0 \leq b < p$, computes $B := g^b \bmod p$, and transmits B to Alice. Thereafter, Alice computes $k = B^a \bmod p$ and Bob computes $k = A^b \bmod p$; in the end, their shared secret is $g^{ab} \bmod p$. The values computed by Alice and Bob are the same thanks to Fermat's Little Theorem: for every integer x and a prime p, $x^p \bmod p = 1$. This protocol is considered secure because the following problem, called the *discrete logarithm problem*, is computationally intractable: given g, A, p, find a (which will be unique when p is a prime) such that $g^a \bmod p = A$. Essentially, the attacker has to try all possible values of a, of which there are $\approx 2^k$ many.

3.3 Asymmetric-key cryptography

In this case, we have two different but mathematically related keys: k_e is the *public key*, used for encryption, and $k_d \neq k_e$ is the *private key*, used for decryption. The main advantages of asymmetric-key cryptography are that (i) it solves the key distribution problem (no need to establish secrets at the beginning of a communication, as the public keys can be shared openly), and (ii) it provides authentication/digital signature. The main disadvantage is that it is slow compared to symmetric-key algorithms. Hence asymmetric-key protocols are often used to establish a symmetric *session key*.

We discuss RSA, the most classical asymmetric cryptographic algorithm, named after its inventors Rivest, Shamir, and Adleman. Recall that Diffie-Hellman algorithm was based on computational difficulty of the discrete logarithm problem. RSA, on the other hand, is based on the computational difficulty of factoring large numbers.

In the context of RSA, everybody has a public key k_e and a private key k_d. We generate such a pair as follows. First generate two large prime numbers p, q (each around 1000-2000 bits long), and set $n = pq$. Then compute $\varphi = (p-1)(q-1)$. Next, choose $1 < e < \varphi$ such that $\gcd(e, \varphi) = 1$. In particular, we can choose e to be any prime in $(1, \varphi)$. In most implementations of RSA, e is chosen to be 65537. Finally, use the extended Euclidean algorithm (which is computationally very efficient) to find d such that $e \cdot d \bmod \varphi = 1$. Then the public key is (n, e) and the private key is (n, d). Note that if we could factor n as $n = pq$, then we could immediately compute the private key from the public key.

We make (n, e) openly available, and when someone wants to send a message m to us, they encrypt it as follows. We suppose that, when interpreted as a number, $m < n$; otherwise we have to send multiple messages. The ciphertext (which is sent to us) is simply the (binary representation of) $c := m^e \bmod n$. (We mention that, due to our choice of e, every different $0 < m_1, m_2 < n$ are mapped to different ciphertexts, i.e. $m_1^e \bmod n = m_2^e \bmod n$ only when $m_1 = m_2$.) To decrypt c, we simply compute $m = c^d \bmod n$. This works because of laws of modular arithmetic.

Still, RSA is never used in practice in its raw form described above. The reason is that it lacks randomness: it is possible, for example, to attack signatures of small/common messages because the same messages are always signed/encrypted in exactly the same way. Modern implementations of RSA never encrypt/sign only the message m, but rather a transformation of m that involves a random string (which is then transmitted along the ciphertext/signature).

3.4 Digital signatures

The reverse of the process described above can be used to implement *digital signatures*. Suppose I need to sign a message m. First, I compute the hash of m, written h. Then I compute the signature $s = h^d \bmod n$, and transmit it along m and h. Upon receiving (m, h, s), the recipient (i) calculates the hash h' of m, and compares it with h, and (ii) computes $s^e \bmod n$ and compares the result with h. If everything matches, then it means that the message was undeniably sent by me. This protocol provides *authentication* and *non-repudiation*. (We mention that symmetric-key protocols

do not provide non-repudiation, because the key is known to two people: thus either person can plausibly claim that the action was performed by the other.) In summary, RSA works because by the properties of p, q, e, d, for any m we have that

$$(m^e \bmod n)^d \bmod n = m = (m^d \bmod n)^e \bmod n,$$

and the problem of determining the value of d only knowing n and e is computationally infeasible.

How are public keys bound to real-world identities? We will discuss this in detail later, but the short answer is that, when I claim to someone to be entity X, I present a certificate digitally signed by a trusted *certificate authority* (CA). The CAs verify the identity (using various means: for example, to verify that you own a domain name they can ask you to create a certain page under your domain name) when producing the certificate, but also have the power to revoke a certificate when, for example, a private key gets compromised.

3.5 Discussion and homework

In AES-CTR, what if we do not generate a nonce? That is, what would happen if we use just $\mathrm{AES}(1) \,||\, \mathrm{AES}(1) \,||\, \cdots$ as the key stream?

Is it better to every time establish a session key using Diffie-Hellman, or exchange messages using RSA? The former provides *forward secrecy*, whereas the latter does not. Forward secrecy means that if a key (e.g. the private key in RSA) gets compromised, then the attacker (who can store previous messages) is not able to decrypt past messages. In case of Diffie-Hellman, each session has its own key; hence a loss of one key does not compromise messages from other sessions.

However, unauthenticated Diffie-Hellman is weak to the *man-in-the-middle attack*. Here, Alice and Bob think that they are establishing a session key, but instead end up each establishing a session with Eve, who can then relay and modify the messages between Alice and Bob as she pleases. The modern way is to establish the *ephemeral* Diffie-Hellman key over an (RSA-)authenticated communication channel. Question: Suppose one party generates a session key and sends it to the other one using RSA encryption. Is this a good protocol? Why bother with doing both RSA and Diffie-Hellman?

RSA is not actually considered the strongest nor the most modern asymmetric-key algorithm: *elliptic curve cryptography* is considered stronger, and needs significantly smaller key sizes. However, both families of algorithms are insecure against *quantum computers*. Completely new and even stronger algorithms have been proposed that will remain secure even if we manage to develop practical quantum computers; the area that studies this is called *post-quantum cryptography*.

Homework. Take a look at `https://nvlpubs.nist.gov/nistpubs/FIPS/NIST.FIPS.197-upd1.pdf` for the official specification of the AES algorithms. Investigate the modes of AES that we did not discuss. Look up other symmetric-key encryption protocols (e.g., ChaCha20) and where they are used. Investigate how Diffie-Hellman key exchange algorithm is implemented in practice, e.g. how large prime numbers are found and how $g^a \bmod p$ is efficiently computed. Look up the following attack modes: ciphertext-only attack (COA), known-plaintext attack (KPA), chosen-plaintext attack (CPA), chosen-ciphertext attack (CCA). (Note: when discussing security of protocols, it is common to assume at least a CPA and usually a CCA attacker.)

3.6 Quick one: message authentication codes (MAC)

We saw that AES-GCM computes, in addition to the ciphertext, a tag. This is an example of a MAC. Encryption provides confidentiality, but it does not provide integrity. MACs, on the other hand, provide integrity and authentication (but no confidentiality on their own without accompanying encryption). MACs can be constructed from a hash function (HMAC; hash the key and the message together), or a cipher (CMAC; like a block hash function, but instead of a one-way function, use a block cipher internally).

When using MAC together with encryption, the correct way is to first encrypt the message m to get the ciphertext c, and then compute the MAC for c, not the other way around. This way, the receiver first checks the MAC for the whole message. If the check fails, the she does not decrypt the message at all: this is a defence against chosen-ciphertext attacks.

We mention that confidentiality + integrity is still not everything: the attacker can store a valid combination of a ciphertext and a MAC, and send it (for the first time or again) at a later time. This is called a *replay attack*. To prevent this, messages can include, for example, the date and time or the

session number. The term for being safe from a replay attack is *freshness*: it means that the message is fresh and not from the past.

4 Access control in operating systems

Different users of the same system share its computing resources, which include memory, the file system, computing power, networks, peripherals etc. The term *authorisation* refers to what each user is allowed to do. In this section, we discuss access control in a Linux system.

The directory `/` is called the *root* of a (Linux-type) file system. The whole file system is organised under `/`. Additionally, each user has the *home directory* `/home/username`, which can also be accessed using the shorthand $\sim$. (In MacOS, despite it being Linux-based, the home directories of all users are stored in `/Users`.) There is a special user called the *root user*, which typically has unlimited power. Usually, no actual (human) user is actually logged in as the root user: rather, there is a command called `sudo` (which means *superuser do*) through which a select set of users can temporarily give themselves root user privileges. (An account with full `sudo` privileges can also configure what other users can do using `sudo`.) For example, the first user of a Linux system (typically the system administrator) can do everything that the root user can do using `sudo`. If you are truly logged in as the root user (which usually does not happen or is not allowed due to security reasons), then you don't need `sudo`.

The traditional access control model of a Linux system is called *Discretionary Access Control* (DAC). Here we have users and groups (which are collections of users with common permissions). Each running process has a user ID (UID), which identifies the user to which the process belongs, as well as one or more group IDs (GIDs), which record all groups to which the owner of the process belongs. Similarly, each file has an associated UID (the identity of the owner), a GID (the group of the file), as well as a list of permission for the owner, the group, and everyone else. The things a process is authorised to do to a file is determined by checking the UIDs and the GIDs associated with the process and the file in question.

The permissions associated with a file are described by a 10-letter string $x_1 \cdots x_{10}$, where each letter except the first one is one of `r` (read), `w` (write, i.e. modify), `x` (execute, i.e. run), or `-` (no permissions). Here

1. x_1 describes the type of the file;
2. $x_2x_3x_4$ describes the permissions of the owner, e.g. $x_2x_3x_4$=`rwx` means the owner can read, write, and execute;
3. $x_5x_6x_7$ describes the permissions of the users belonging to the group of the file; $x_5x_6x_7$ describes the permissions of everyone else.

Any user with root privileges can create a new user, delete an existing user, change their password/groups, view the contents of all files, control the network access of other users etc.. (Having root privilege means having UID zero, which is not the same as having `sudo` access: what each user can do using the `sudo` command can be configured by the root user.) It is possible to restrict what the root users can do using *Mandatory Access Control* (MAC).

A user with root privileges can also *audit* other users: this includes reviewing all of their activity, including logins, commands executed etc. See `auditid`.

5 Basics of the internet

The internet is a distributed system for exchanging information that is not owned by a central body. Understanding its architecture as well as the protocols it uses is crucial for ensuring computer security.

5.1 IP addresses

An *IP* (short for *internet protocol*) address is the address of a device connected to the internet. We will discuss *IPv4* and *IPv6*. An IPv4 address is of the form x.x.x.x, where each x is a number between 0 and 255 inclusive. Thus there are $255^4 = 2^{32}$ possible IPv4 addresses. It turns out that there are more devices connected to the internet (these are called *hosts*) than possible IPv4 addresses; we will

discuss this shortly. The solution for the future is IPv6, which uses 128-bit addresses (and thus has 2^{128} possible addresses). All the important protocols that I know support both IPv4 and IPv6.

The solution to the scarcity of IPv4 addresses is that not everyone gets a fixed IPv4 address. We have *public addresses* and *private addresses*. A public address is globally unique, whereas the private addresses (which are always in the ranges 10.0.0.0–10.255.255.255, 172.16.0.0–172.31.255.255, 192.168.0.0–192.168.255.255) are re-used. Each internet service provider (ISP) has a set of public IPv4 addresses that it owns. End users first connect to their local area network (LAN), where devices are identified by private IP addresses. In the LAN, there are some devices that are connected to the *wide-area network* (WAN), i.e. the "true internet". These devices (which could be just your router) have both a public and a private IP address, and perform *network address translation* (NAT). Essentially, to communicate with the outer world, you send your packets to the NAT box using LAN, and it forwards them to their address, returning the packets from the outer world (that were addressed to you) back to you. Formally, a NAT box translates a private IP address + a *port number* (you can think of an IP address as the address of a building, and the port number as the apartment number) to a public IP address (namely, its own public IP address) + a port number. Because of NAT, public IP addresses are not routable, meaning that you cannot send a message to someone's public IP address. Hence establishing communication without either party having a public IP address is actually not completely trivial.

5.2 The network layer

Communication over the internet happens at different levels of abstraction: over the cable at the physical level and between programs running on different computers at the highest level. In fact, the *OSI network model* describes 7 levels of abstraction: each level uses the levels below it as a black box. IP addresses are at Level 3, called the *network layer*.

In the network layer, the hosts exchange messages called *packets*. Each packet has, among other things, the source and destination IP addresses, checksums, payload (actual information being transmitted), a description of the payload etc. When receiving a packet, the hosts forward it towards the address of the recipient, unless it is addressed to them: this is called *routing*. On average, a packet gets passed around ≈ 15 different hosts before reaching its destination.

5.3 Client-server architecture

In one computer, we run many different clients, and possibly many different servers. A server is someone who waits for a request from a client, and then sends back a response. Often a server has a fixed IP address. A client, on the other hand, sends a request to a server. Not having a public IP address is generally not a problem for a client. Servers can serve different things: for example, it can serve the HTML files of a website upon request from a browser (a client). Similarly, when trying to open a website, the browser first connects to a *domain name service* (DNS) server, which gives you the IP address of the web server for the domain name that you requested.

5.4 Homework

Study the OSI model. We mentioned that, due to NAT, direct peer-to-peer communication between to hosts neither of which has a public (fixed) IP address is not possible. How do WhatsApp/Telegram etc. work then?

6 Linux (bash) programming

Every file is simply a sequence of bits. You can view it in binary and ASCII (though not every byte corresponds to a printable ASCII character) using `xxd -b filename`. To view the contents in hexadecimal, use `xxd filename`.

In a Linux-based system, the command-line program `sha256sum` computes the `SHA2` hashes (in hexadecimal); see `https://man7.org/linux/man-pages/man1/sha256sum.1.html` for the manual. (In `PowerShell`, you would use `Get-FileHash`.) To compute the hash of a file, use `sha256sum filename`. You can also compute hashes of multiple files: `sha256sum filename1 filename2 filename3` To compute the hash of a string, use `printf string | sha256sum` or `echo -n string | sha256sum`;

this is called *piping*, which chains command-line instructions. You can get the hash of a string `s` by doing `printf s | sha256sum | awk 'print $1'`.

You can use the `--check` (or `-c` for short) option of `sha256sum` to verify checksums. To do this, you first create a file `hashes.txt` (can be called anything) of rows of the form `filename checksum` (with exactly two spaces between the two) where `checksum` is the correct hash of the corresponding file. Then run `sha256sum --check hashes.txt`. This goes over every line in `hashes.txt`, hashes the file called `filename`, and compares the result against `checksum`. Try `printf hello > message.txt; sha256sum message.txt > hashes.txt; sha256sum --check hashes.txt`.

In a Linux system, you can find the hashes of the user passwords (together with the corresponding salts) in the directory `etc/shadow`. (You can find more password information under `etc/passwd`.) You need root permission to access the file: `sudo cat etc/shadow`. In Linux Mint, rather than SHA256, yescrypt is used by default for password hashing.

We move on to `hashcat`, which is a command-line program that efficiently performs (using the GPU of a computer) a large number of hashing operations at the same time. It can be used to "crack" a hash. Fundamentally, it takes a hash y (stored in a file) and a list of candidate words x, and checks for all x whether $f(x) = y$. You can install `hashcat` using `sudo apt install hashcat`. See "General guides" at `https://hashcat.net/wiki/` for documentation. For example, `hashcat -m 1400 -a 0 hashes.txt wordlist.txt` performs a dictionary attack (because we say `-a 0`) using `SHA2` (because we say `-m 1400`) on hashes stored in `hashes.txt` using a list of candidate inputs `wordlist.txt`.

We can perform AES-CTR encryption and decryption using `openssl`. (It does not, however, support AES-GCM or any other AEAD modes.) To encrypt, we use `openssl enc -aes-256-ctr`. It expects (i) the plaintext, either from `stdin` or via the optional argument `-in <filename>`, (ii) a key in the form `-K <256 bits as hex characters>`, and (iii) a 128-bit *initialisation vector* (which has the same function as nonce+block counter) in the form `-iv <128 bits as hex characters>`. It outputs a binary string, which you can redirect to a file using `>`. Alternatively, you can specify an output file using the `-out` option. Decryption works in exactly the same way, but you additionally use the `-d` option. We can use shell variables to store the key and the nonce: for example, we can do `KEY=$(openssl rand -hex 32)` to generate and store 32 random bytes (represented as hex characters), and then access with `"$KEY"`.

One can generate a public-private RSA key pair using `ssh-keygen -t rsa`. Usually it will be stored in the (hidden) folder `~/.ssh` (where the dot means the file is hidden). You can view the contents of this folder using `sudo ls -a ~`. If you do `sudo ls ~/.ssh`, you should see the files `id_rsa` and `id_rsa.pub`. These are the private and public keys, respectively, which can be viewed using `cat`. RSA encryption and decryption (as well as RSA digital signatures) can be performed using `openssl-rsautil` (see `https://docs.openssl.org/master/man1/openssl-rsautl/`) or `openssl-pkeyutil` (more modern, see `https://docs.openssl.org/3.0/man1/openssl-pkeyutl/`). You can use `ssh-keygen` to convert your keys into the PEM format.

You can view the list of all running processes, as well as the associated UIDs/GIDs using the command `ps`. To do the same for a file in the current directory, use `ls -l`, and for a file with absolute path `path`, use `ls -l path`. The command to change permissions of a file is `chmod`. You can create a new user using `sudo adduser username`. The newly created user by default does not have any `sudo` access. To remove an existing user, use `sudo deluser`.

The system variable called `$path` (in Windows it is called `$PATH`) contains a list of directories: when we run `command` $\cdots$, the terminal searches through the directories in `$path` to find the code corresponding to `command`. It is possible to run any file with execution permission, either by typing its absolute path to the shell or by moving to its parent directory and running `./filename`. To run a program from anywhere without needing to know its absolute path, either we add its parent directory to `$path`, or move its code to a directory that is already present in `$path`. Try `echo $path`.